Rapport - Projet 1 : Comment dessiner un arbre

IFT2015 - A26

Membres de l'équipe : Ali Radwan, Gideon Mubenga
Dépôt GitHub : https://github.com/AliRadwan9/projet-2015

1. Organisation du travail d'équipe

(a) Collaboration. Nous avons travaillé ensemble en présentiel et aussi en ligne. Les séances en présentiel ont servi à lire l'énoncé, à élaborer l'algorithme récursif de `tell` et à intégrer nos parties; le travail en ligne a permis de continuer à avancer à distance.

(b) Communication et gestion du code. Le code est partagé sur un dépôt GitHub commun. Chacun y publie ses modifications (commit/push) et récupère celles de l'autre (pull) avant de continuer. Nous nous sommes entendus sur la structure des classes et les signatures des méthodes avant de coder.

(c) Partition des tâches. Les tâches ont été réparties entre nous de façon équitable : chacun a pris en charge une partie de l'implémentation, puis nous avons intégré le tout ensemble.

(d) Conflits. Aucun conflit majeur n'est survenu entre nous.

Temps de travail. Environ 20 heures au total.

2. Explication de l'implémentation

L'implémentation est en Java, dans le paquetage `lindenmayer`. Elle suit la spécification de l'énoncé avec une séparation claire des responsabilités entre les classes.

Tortue. `Turtle` est l'interface fournie (avec `action(String)` par réflexion). `TrueTurtle` est la « tortue à crayon imaginaire » : elle maintient l'état sans rien dessiner.

- Une classe imbriquée `State` encapsule la position et l'angle; le `Point2D` est cloné pour que les déplacements futurs n'écrasent pas l'état sauvegardé.
- Une `ArrayDeque<State>` sert de pile pour `push` et `pop` (préférée à `Stack`, dont les méthodes sont synchronisées).
- `advance()` convertit l'angle de degrés en radians et calcule (x + d cos(a), y + d sin(a)); `turnL` ajoute l'angle unitaire et `turnR` le soustrait.

Tortues de sortie. `EPSTurtle` étend `TrueTurtle` : chaque action met d'abord l'état à jour (appel à `super`), puis écrit du PostScript sur un `PrintStream` : `draw` écrit « x y lineto », `move` « x y moveto », `push` « currentpoint stroke newpath moveto » et `pop` « stroke x y newpath moveto » avec la position restaurée. C'est la tortue utilisée par `Main`. `PostScriptTurtle` est une variante qui calcule elle-même sa boîte englobante et écrit l'en-tête et le trailer. Un autre format (SVG, écran) ne demanderait qu'une nouvelle sous-classe.

Le L-système. `LSystem` étend `AbstractLSystem` et s'appuie sur trois dictionnaires :

- `Map<Character, Symbol>` : garantit une seule instance de `Symbol` par caractère (`computeIfAbsent`);
- `Map<Symbol, String>` : associe chaque symbole à son action de tortue;
- `Map<Symbol, List<List<Symbol>>>` : stocke toutes les règles d'un même côté gauche.

`rewrite(sym)` retourne `null` si le symbole est terminal; sinon il choisit uniformément une règle au hasard avec `rndElement`. Ajouter deux fois la même règle augmente donc sa probabilité. Le générateur est reproductible grâce à la graine (*seed*) de `AbstractLSystem`.

Initialisation JSON. `initFromJson` lit les clés `actions`, `axiom`, `rules` et `parameters`. Elle appelle `setAction`, `setAxiom` et `addRule`, puis `turtle.setUnits(step, angle)` et, si `start` est présent, `turtle.init(...)`. Des valeurs par défaut sont fournies avec `optDouble`.

Inférence récursive. `tell(turtle, seq, rounds)` ne construit jamais la chaîne complète. Pour chaque symbole, `processSymbol` : si `rounds == 0` ou si le symbole est terminal, il exécute `tell(turtle, sym)` et ajoute la position de la tortue à la boîte englobante (`Rectangle2D.add(Point2D)`); sinon, il s'appelle récursivement sur chaque symbole de l'expansion avec `rounds - 1`. La mémoire utilisée est donc proportionnelle

à la profondeur de récursion, et non à la longueur de la chaîne dérivée.

Application. `Main` lit le fichier JSON (argument 1) et le nombre d'itérations (argument 2), initialise le système et la tortue, puis écrit l'en-tête EPS avec `%%BoundingBox: (atend)` et le chemin initial `newpath x y moveto`. Après `lsystem.tell(...)`, il ferme le tracé avec `stroke` puis écrit `%%Trailer` et le `%%BoundingBox` final (`floor` pour les minimums, `ceil` pour les maximums). La sortie va sur `System.out` :

```
java -jar lindenmayer.jar test/buisson.json 5 > buisson5.eps
```

3. Librairies utilisées

- **JSON-java (**`org.json`**)** : librairie externe pour lire le fichier JSON (`JSONObject`, `JSONTokener`, `JSONArray`).
- `java.util.Locale` **(**`Locale.US`**)** : ce n'est pas une librairie externe mais une classe standard du JDK, qui gère la conversion entre les conventions américaine (US) et canadienne (CA). Sur un système en français canadien, `printf` écrit les décimaux avec une virgule (2,50), ce qui est invalide en PostScript. Tous nos `printf` utilisent `Locale.US` pour toujours obtenir un point décimal (2.50) et un fichier EPS valide, quelle que soit la langue du système. Nous limitons aussi à 2 décimales (`%.2f`) pour garder le fichier compact.
- `java.awt.geom` (`Point2D`, `Rectangle2D`) : géométrie des positions et de la boîte englobante.